

Análise de Test Smells e Refatoração

Disciplina: Teste de Software

Gustavo Pimentel Carvalho Costa - Matrícula: 833151

Projeto: test-smelly

1. Resumo dos Resultados

Após a refatoração, a suite de testes apresenta:

Métrica	Antes	Depois
Total de testes	4 (+ 1 skipped)	14 (+ 1 skipped)
ESLint errors (smelly)	4 errors, 2 warnings	0 errors, 0 warnings
ESLint warnings (clean)	-	0 warnings
Testes passando	4 passed	14 passed
Smells identificados	5	0 (refatorados)

2. Análise de Smells Identificados

#	Smell	Localização	Regra ESLint
1	Eager Test	deve criar e buscar usuário	N/A (manual)
2	Conditional Logic	deve desativar usuários	jest/no-conditional-expect
3	Error Handling	deve falhar menor idade	N/A (manual)
4	Fragile Test	deve gerar relatório	N/A (manual)
5	Skipped Test	lista vazia	jest/no-disabled-tests

3. Detecção por Ferramenta

Resultado do ESLint no arquivo smelly:

test/userService.smelly.test.js

44:9 error Avoid calling expect conditionally jest/no-conditional-expect

46:9 error Avoid calling expect conditionally jest/no-conditional-expect

49:9 error Avoid calling expect conditionally jest/no-conditional-expect

73:7 error Avoid calling expect conditionally jest/no-conditional-expect

77:3 warning Tests should not be skipped jest/no-disabled-tests

77:3 warning Test has no assertions jest/expect-expect

Total: 4 errors, 2 warnings

Após refatoração:

test/userService.clean.test.js - 0 errors, 0 warnings

4. Processo de Refatoração

4.1 Error Handling Anti-Pattern (Smell 3)

ANTES (try/catch silencioso):

```
test('deve falhar ao criar usuário menor de idade', () => { try { userService.createUser('Menor', 'menor@email.com', 17); } catch (e) { expect(e.message).toBe('O usuário deve ser maior de idade. '); } });
```

PROBLEMA: Se a validação for removida, o teste passa silenciosamente sem nenhuma asserção executada.

DEPOIS (expect().toThrow()):

```
test('deve lançar erro ao criar usuário menor de idade', () => { expect(() => { userService.createUser('Menor', 'menor@email.com', 17); }).toThrow('O usuário deve ser maior de idade. '); });
```

SOLUÇÃO: O expect().toThrow() garante que a exceção será lançada. Se a validação for removida, o teste falha imediatamente.

4.2 Conditional Logic in Test (Smell 2)

ANTES (loop + if com expect condicional):

```
for (const user of todosOsUsuarios) { const resultado = userService.deactivateUser(user.id); if (!user.isAdmin) { expect(resultado).toBe(true); } else { expect(resultado).toBe(false); } }
```

PROBLEMA: expect dentro de if pode não executar para todos os casos. Um mutante que inverta a lógica pode sobreviver.

DEPOIS (testes explícitos separados):

```
test('deve desativar usuário comum e retornar true', () => { const usuario =
userService.createUser('Comum', 'comum@email.com', 30); const resultado =
userService.deactivateUser(usuario.id); expect(resultado).toBe(true); }); test('deve recusar
desativação de admin e retornar false', () => { const admin = userService.createUser('Admin',
'admin@email.com', 40, true); const resultado = userService.deactivateUser(admin.id);
expect(resultado).toBe(false); });
```

SOLUÇÃO: Cada teste verifica um caminho específico. Sem lógica condicional, todo comportamento é explícito.

5. Estrutura AAA nos Testes Refatorados

Todos os testes seguem o padrão Arrange-Act-Assert:

```
test('deve retornar o usuário correto pelo id', () => { // Arrange const criado =
userService.createUser('Bob', 'bob@email.com', 30); // Act const encontrado =
userService.getUserById(criado.id); // Assert expect(encontrado.nome).toBe('Bob'); });
```

6. Conclusão

A refatoração seguindo o padrão AAA (Arrange-Act-Assert) eliminou os 5 smells identificados: 1. Eager Test → separados em testes unitários 2. Conditional Logic → testes explícitos sem loop/if 3. Error Handling → expect().toThrow() em vez de try/catch 4. Fragile Test → verificação de comportamento em vez de string exata 5. Skipped Test → implementado O ESLint como ferramenta de análise estática complementa o teste manual, detectando 2 dos 5 smells automaticamente (conditional-expect e disabled-tests). Os outros 3 requerem revisão de código. A qualidade da suite de testes melhorou significativamente: 14 testes passando, zero warnings ESLint, e código legível que especifica comportamento ao invés de apenas executar.